Advanced Database Systems

GIN400 – School of Engineering

Chapter 2
Query Optimization

Dr. Joseph Zalaket

1

Outline

- Query Processing and Optimization
- Cost estimation
  - Cost estimation for Selection
  - External Sort-Merge algorithm
  - Cost estimation for Join
  - Cost estimation for other operations
  - Combining Operations: Materialization and Pipelining
- Heuristics optimization

Dr. Joseph Zalaket

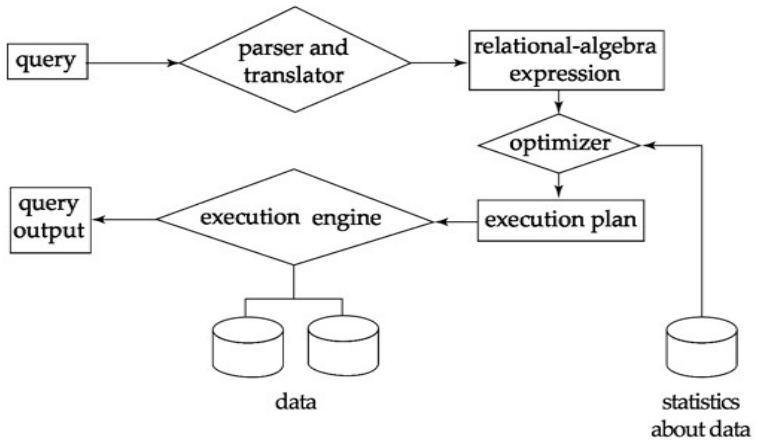
2

Query Processing and Optimization

- ***Query processing:*** consists of transforming a high level query (e.g. SQL) into an equivalent low-level query (in relational algebra).
- In relational algebra we write a sequence of operations to specify retrieval request (procedural language).
- Problem: A relational calculus (e.g. in SQL) query may have many equivalent transformation into relational algebra.
- The goal is to select the execution strategy that minimizes resource consumption.
- ***Query Optimization:*** Amongst all equivalent evaluation plans choose the one with lowest cost.
  - Cost is estimated using statistical information from the database catalog
    - e.g. number of tuples in each relation, size of tuples, etc.

3

Query Processing and Optimization



4

SQL to Relational Algebra

- **Query block:** the basic unit that can be translated into the algebraic operators and optimized.
- A query block contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clause if these are part of the block.
- Nested queries within a query are identified as separate query blocks.
- Aggregate operators in SQL must be included in the extended algebra.
- The query optimizer chooses an execution plan for each block.

Dr. Joseph Zalaket

5

Cost estimation

- Cost estimation: Statistics, maintained in system catalogs, used to estimate cost of operations and result sizes.
  - Considers combination of CPU and I/O costs.
- **Catalog Information Used in Cost Functions**
  - Information about the size of a file
    - **number of records (cardinality) (r),**
    - **record size (R),**
    - **number of blocks (b)**
    - **blocking factor (bfr)**
  - Information about indexes and indexing attributes of a file
    - **Number of levels (x)** of each multilevel index
    - **Number of first-level index blocks (b_{I1})**
    - **Number of distinct values (d)** of an attribute
    - **Selectivity (sl)** of an attribute ($0 \leq sl \leq 1$)= # of records satisfying the condition / r (cardinality)
    - **Selection cardinality (s)** of an attribute. (s = sl * r) = # of records satisfying the condition

Dr. Joseph Zalaket

6

Cost estimation for SELECTION

- **S1. Linear search (brute force) approach:** Scan each file block and test all records to see whether they satisfy the selection condition.
  - Cost estimate (number of disk blocks scanned) C= b
  - If selection is on a key attribute, C = (b/2) if the record is found stop on finding record) ; otherwise C = b.
  - Linear search can be applied regardless of
    - selection condition, or
    - ordering of records in the file, or
    - availability of indices
- **S2. Binary search**: Applicable if selection is an equality comparison on the attribute on which file is ordered.
  $C = \log_2 b + \lceil (s/bfr) \rceil - 1$
  For an equality condition on a unique (key) attribute,
  $C = \log_2 b$
- **S3. Using a primary index to retrieve a single record**
  C = x + 1;

Dr. Joseph Zalaket

7

Cost estimation for SELECTION

- **S4. Using a hash key to retrieve a single record**
  C = 1 for static hashing
  C = 2 for extendable hashing;
- **S5. Using an ordering index to retrieve multiple records**. For a comparison condition such as >, ≥, <, or ≤ on a key field with an ordering index:
  C= x+(b/2) rough estimation
- **S6. Using a clustering index to retrieve multiple records** for an equality condition
  $C = x + \lceil (s/bfr) \rceil$
- **S7. Using a secondary (B^+-tree (unclustered)) index:**
  a) For an equality comparison, C = x + s;
  b) For a comparison condition such as >, <, >=, or <=,
  $C = x + (b_{l1}/2) + (r/2)$

Dr. Joseph Zalaket

8

Selectivity

- Selectivity (sl) = $\frac{\#\text{ of records satisfying the condition}}{\text{total }\#\text{ of records}}$

  - Estimates of selectivities =

  – (1) key attribute $\frac{1}{r}$

  – (2) nonkey attribute $\frac{1}{d}$

- Where, d = number of distinct values for the attribute in r(n)

Dr. Joseph Zalaket

9

Cost estimation for SELECTION

- **S8. Conjunctive selection:**
  Use either S1 or one of the methods S2 to S7 to solve.
  For the latter case, use one condition to retrieve the records and then check in the memory buffer whether each retrieved record satisfies the remaining conditions in the conjunction.
- **S9. Conjunctive selection using a composite index**:
  Same as S3, S6 or S7a, depending on the type of index.

Dr. Joseph Zalaket

10

Cost estimation examples

- **EMPLOYEE(SSN, FNAME, NAME, BDATE, ADDRESS, SEX, SALARY, SUPERSSN, DNO)**
- **DEPARTMENT (DNUMBER, DNAME, MGRSSN,MGRSTARTDATE)**
  - r_E =10,000 records,
  - b_E =2000 disk blocks
  - bfr_E= 5 records / block
- Access paths:
  1. Primary index on key attribute SSN: $\mathbf{x}_{SSN}$ = 4, $\mathbf{s}_{SSN}$= 1
  2. Secondary index on DNO: $\mathbf{x}_{DNO}$ = 2 , $\mathbf{b}_{I1DNO}$ = 4, $\mathbf{d}_{DNO}$ = 125
     $\mathbf{s}_{DNO}$ = (10,000 / 125) = 80
  3. Secondary index on SEX: $\mathbf{x}_{SEX}$=1, $\mathbf{d}_{SEX}$=2
     $\mathbf{s}_{SEX}$ = (10,000 / 2) = 5000

Cost estimation examples

- $\sigma_{SSN='123456789'}$**(EMPLOYEE)**
  - Brute force (S1) : C=(b_E / 2) =(2000 / 2) = 1000
  - Binary search (S2): C =$\log_2 b_E$ =10.96 ≈ 11
  - Primary index (S3): C= x_{SSN} + 1 = 4 + 1 =5
- $\sigma_{DNO > 5}$ **(EMPLOYEE)**
  - Brute force (S1) : C = b_E = 2000
  - Secondary index (S7) : C= x_{DNO}+ (b_{I1DNO} / 2) + (r_E / 2) = 2 + (4 / 2) + (10,000 / 2) = 5004

    Rk: As a rule, it is probably cheaper to simply scan the entire table (instead of using an unclustered index) if over 5% of the tuples are to be retrieved.

Cost estimation examples

- **$\sigma_{DNO = 5}$ (EMPLOYEE)**
  - Brute force (S1) : C= b_E = 2000
  - Secondary index (S7) :C = x_{DNO}+ s_{DNO}= 2 + 80 = 82
- **$\sigma_{DNO =5 \text{ AND } SEX=F}$ (EMPLOYEE)**
  - Brute force (S1) : C= b_E = 2000
  - Condition DNO=5 (S7-a):
    - C = X_{DNO}+ S_{DNO} = 2 + 80 = 82
  - Condition SEX='F' (S7-a):
    - C = X_{SEX}+ S_{SEX}= 1+ 5000 = 5001

  So, the condition (DNO=5) will be used first to retrieve the records, and the remaining part of the conjunctive condition (SEX='F') is checked for each selected record after it is retrieved into memory.

13

External sorting

- **External Sort-Merge Algorithm** refers to sorting algorithms that are suitable for large files of records on disk that do not fit entirely in main memory.
- The sort process is at the heart of many relational operations:
  - ORDER BY.
  - Sort-merge algorithms for JOIN and other operations (UNION, INTERSECTION).
  - Duplicate elimination algorithms for the PROJECT operation (DISTINCT).

14

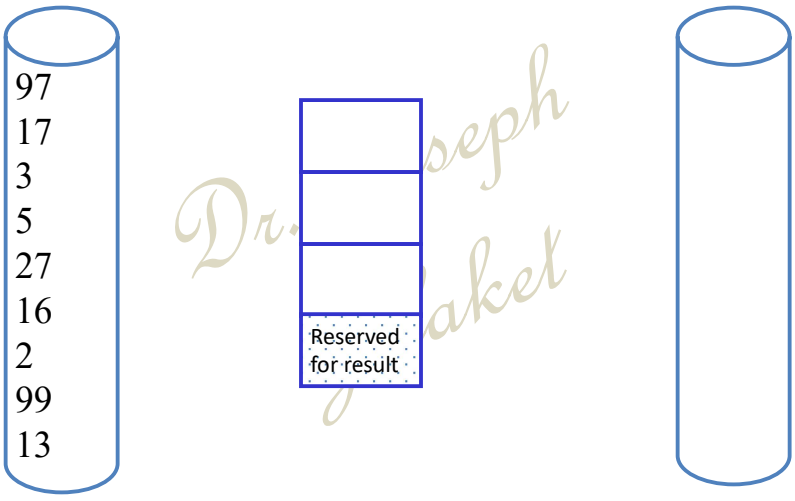
Sort-Merge Algorithm

- Phase I: Create Runs
  1. Load allocated buffer space with tuples
  2. Sort tuples in buffer pool
  3. Write sorted tuples (run) to disk
  4. Goto Step 1 (create next run) until all tuples processed
- Phase II: Merge Runs
  - Use priority heap to merge tuples from runs (start by choosing the first from each run)
- Special cases
  - buffer size (n_B)>= b (size of the file): no merge needed)
  - buffer size (n_B) < sqrt(b): multiple merge phases necessary)

Dr. Joseph Zalaket

15

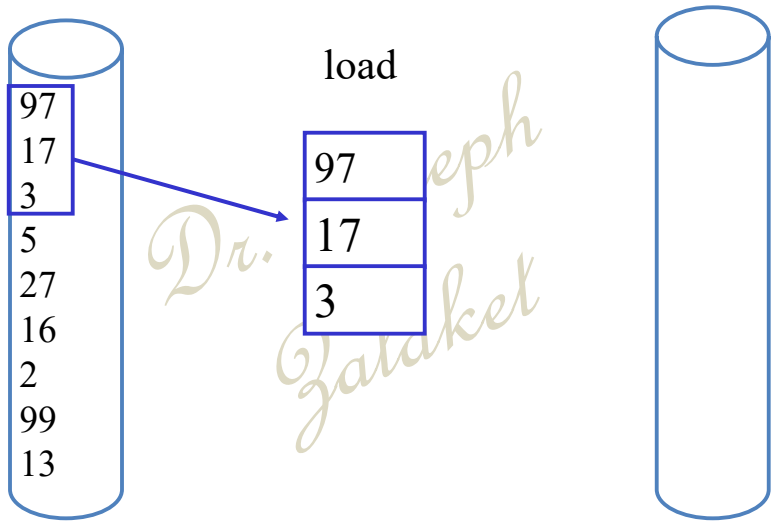
Sort-Merge Algorithm



Dr. Joseph Zalaket

16

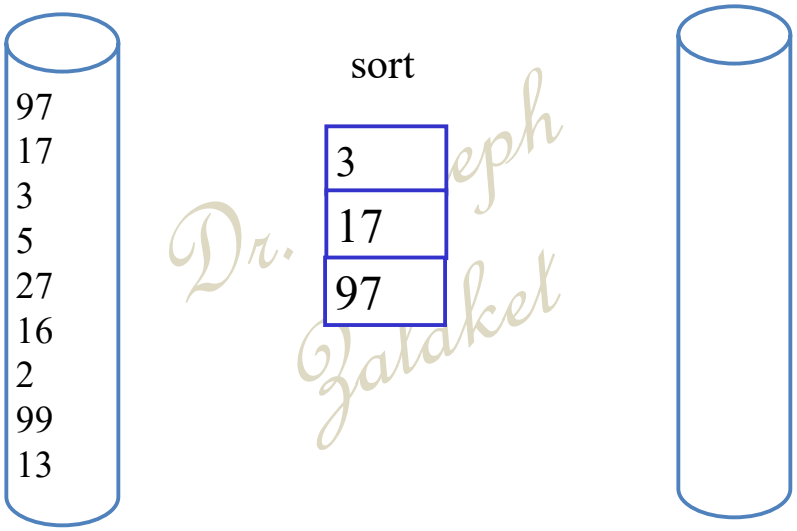
Sort-Merge Algorithm



Dr. Joseph Zalaket

17

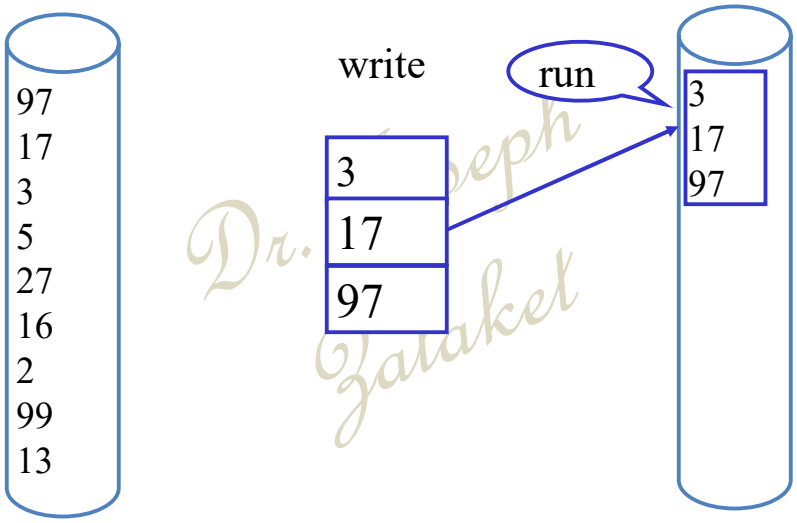
Sort-Merge Algorithm



Dr. Joseph Zalaket

18

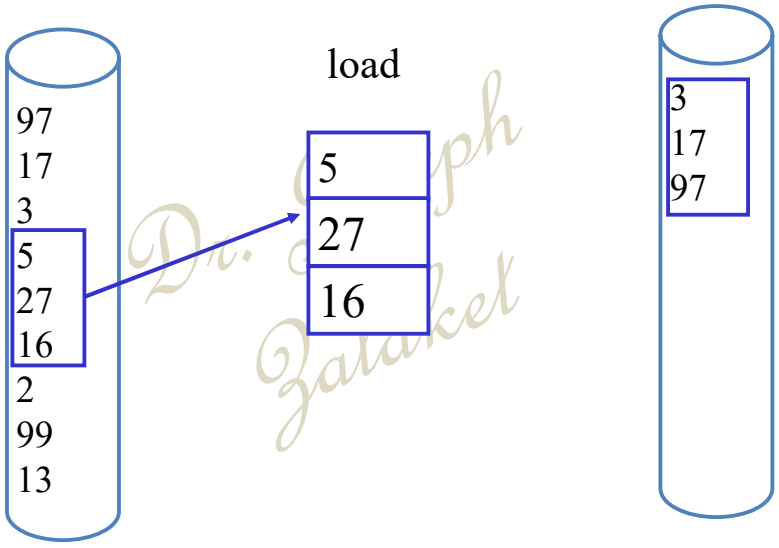
Sort-Merge Algorithm



Dr. Joseph Zalaket

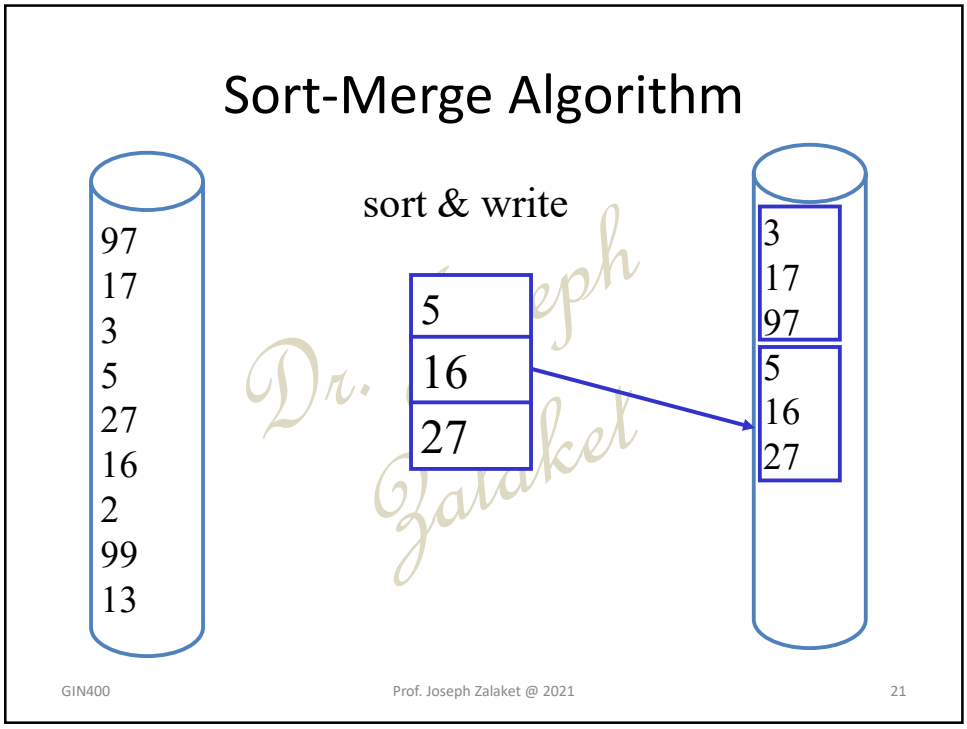
19

Sort-Merge Algorithm

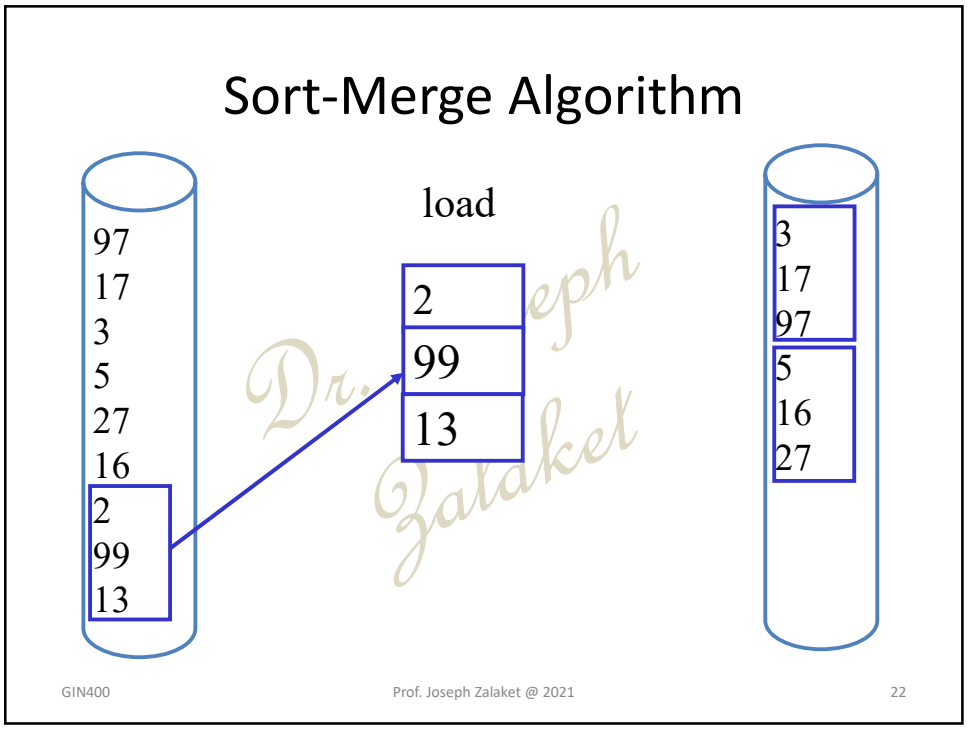


Dr. Joseph Zalaket

20

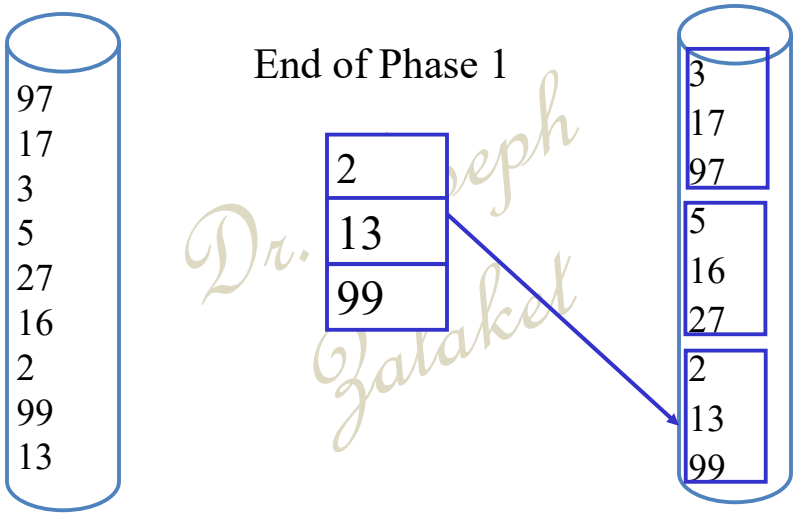

Sort-Merge Algorithm
sort & write
97
17
3
5
27
16
2
99
13
5
16
27
3
17
97
5
16
27
GIN400
Prof. Joseph Zalaket @ 2021
21

21


Sort-Merge Algorithm
load
97
17
3
5
27
16
2
99
13
2
99
13
3
17
97
5
16
27
GIN400
Prof. Joseph Zalaket @ 2021
22

22

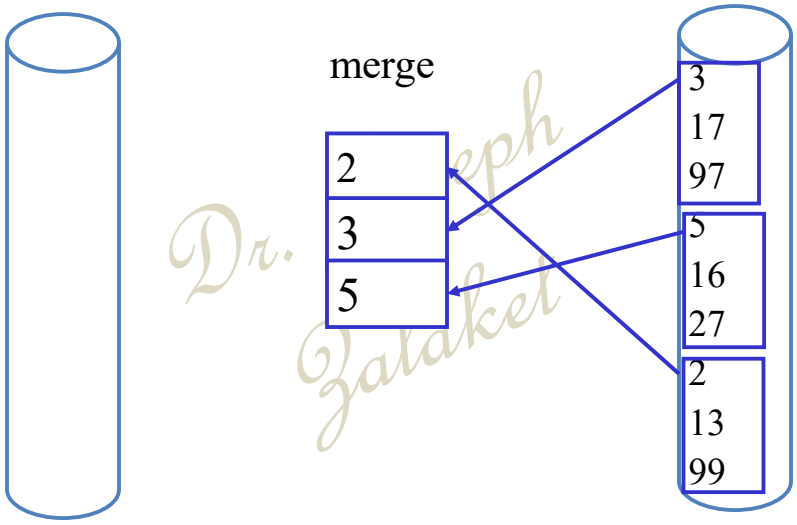
Sort-Merge Algorithm



Dr. Joseph Zalaket

23

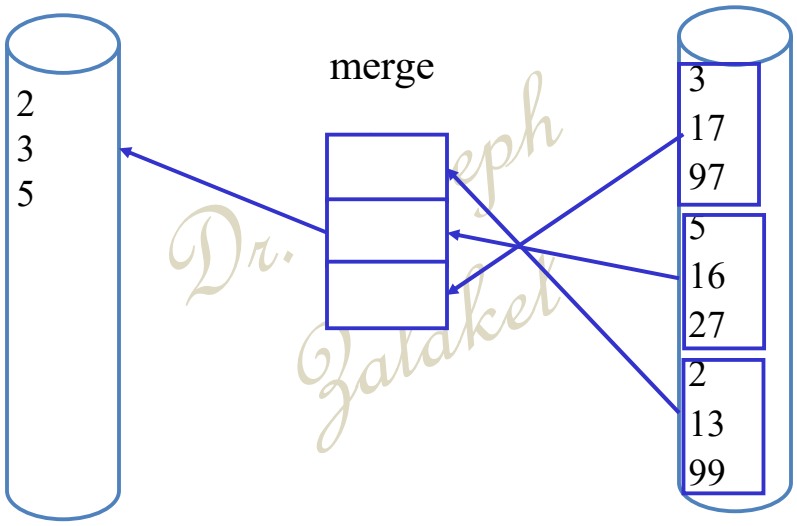
Sort-Merge Algorithm



Dr. Joseph Zalaket

24

Sort-Merge Algorithm



25

Sort-Merge cost

- b: number of file blocks;
- n_B: available buffer space;

- Sorting phase: (number of initial runs) $\mathbf{n_R = \lceil (b/n_B) \rceil}$
  Example-a: n_B =5 blocks, b=1024 blocks $\Longrightarrow n_R = \lceil (1024/5) \rceil$ = 205 runs (of size 5 except the last run of size 4)
- Merging phase: $\mathbf{d_M = Min\ (n_B\text{-}1, n_R)}$; $\mathbf{n_P = \lceil (\log_{dM}(n_R)) \rceil}$
  - Where, d_M: degree of merging (number of runs that can be merged together in each pass) and n_P: number of passes.
  - Example-a: $d_M = 4 \Longrightarrow n_P = \lceil (\log_4(205)) \rceil = 4$
  - $d_M = 4$ that means:
    - pass1 205 records merged into 52, pass2 52 into 13, pass3 13 into 4, pass4 4 into 1
- **Cost = 2b * (1+ n_P)**
  or directly: Cost = $2b * \lceil 1 + (\log_{nB-1} \lceil (b/n_B) \rceil) \rceil$, if $n_B\text{-}1 < n_R$
  - Remark : 2b because each record is accessed twice once for reading into memory and once for writing back to the disk after sorting
  - Example-a: Cost= 2048(1+4)= 10240

26

Implementing joins

- two–way join: a join on two files
  - e.g. $R \bowtie_{A=B} S$
- multi-way joins: joins involving more than two files
  - e.g. $R \bowtie_{A=B} S \bowtie_{C=D} T$

Dr. Joseph Zalaket

27

Nested-loop join

- J1. **Nested-loop join** (brute force): For each record t in R (outer loop), retrieve every record s from S (inner loop) and test whether the two records satisfy the join condition t[A] = s[B].
  - Worst case: Cost = $b_r * b_s + b_r + b_{RES}$
  - Best case: Cost = $b_r + b_s + b_{RES}$
    Where, b_{RES} is the cost of writing the result file
- Improvements to nested-loop join algorithm: use (n_B - 2) disk blocks as blocking unit for outer relations, where n_B = memory buffer size used for join; use remaining two blocks one to buffer inner relation and the second to the output result.
  - Cost = $\lceil b_r / (n_B - 2) \rceil * b_s + b_r + b_{RES}$

Dr. Joseph Zalaket

28

Nested-loop join

- Example: EMPLOYEE $\bowtie_{DNO = DNUMBER}$ DEPARTMENT
  - n_B = 7 blocks (buffers)
  - DEPARTMENT : r_D =50 records, b_D = 10 disk blocks
  - EMPLOYEE: r_E =6000 records , b_E = 2000 disk blocks

1. EMPLOYEE used for outer loop

$$Cost = b_E + (\lceil b_E/(n_B - 2) \rceil * b_D)$$

$$= 2000 + (\lceil (2000/5) \rceil * 10) = 6000 \quad block \quad accesses$$

2. DEPARTMENT used for outer loop

$$Cost = b_D + (\lceil b_D/(n_B - 2) \rceil * b_E)$$

$$= 10 + (\lceil (10/5) \rceil * 2000) = 4010 \quad block \quad accesses$$

- Remark: second is better but both are better than (1) $b_E * b_D + b_E$=22000 and (2) $b_D * b_E + b_D$=20010 (without improvement)

Single-loop join

- J2. **Single-loop join** (Using an access structure to retrieve the matching records): If an index (or hash key) exists for one of the two join attributes — say, B of S — retrieve each record t in R, one at a time, and then use the access structure to retrieve directly all matching records s from S that satisfy s[B] = t[A].
  - Cost = $b_r + r_r * c_i + b_{RES}$
    - Where c_i is the cost of traversing index and fetching all matching *s* tuples for one tuple of *r*
    - c_i can be estimated as cost of a single selection on *s* using the join condition.
- If indices are available on join attributes of both *R* and *S,* use the relation with fewer tuples as the outer relation.

Single-loop join

- Calculation of index traversing cost c_i : Cost = b_r + $r_r * c_i$
  - For a secondary index,
    Cost= $b_R + (r_r * (x_B + s_B)) + b_{RES}$
    For a clustering index,
    Cost = $b_R + (r_r * (x_B + (s_B/bfr_B))) + b_{RES}$
  - For a primary index,
    Cost = $b_R + (r_r * (x_B + 1)) + b_{RES}$
  - If a hash key exists for one of the two join attributes — B of S
    Cost = $b_R + (r_r * h) + b_{RES}$
    where $h \geq 1$ is the average number of block accesses to retrieve a record, given its key hash value.

Single-loop join

- Example: DEPARTMENT $\bowtie_{MGRSSN=SSN}$ EMPLOYEE
  - Suppose primary index exists on SSN of EMPLOYEE : $X_{SSN} = 4$
  - and a secondary index on MGRSSN of DEPARTMENT : $X_{MGRSSN}=2$

1. Retrieve each EMPLOYEE record and then use the index on MGRSSN of DEPARTMENT

$$Cost = b_E + (r_E * (X_{MGRSSN} + 1)) = 2000 + 6000 \times 3 = 20000 \quad block \quad accesses$$

2. Retrieve each DEPARTMENT record and then uses the index on SSN of EMPLOYEE

$$Cost = b_D + (r_D * (X_{SSN} + 1)) = 10 + (50 \times 5) = 260 \quad block \quad accesses$$

- Remark: here the 50 departments will be joined and only 50 from 6000 employees (who are managers)

Sort-merge join

- J3. **Sort-merge join:** If the records of R and S are physically sorted (ordered) by value of the join attributes A and B, respectively. Both files are scanned in order of the join attributes, matching the records that have the same values for A and B. In this method, the records of each file are scanned only once each for matching with the other file.
  - Cost= $b_R + b_S + b_{RES}$
- If records of R and S are unsorted
  - Apply external sort-Merge algorithm to R and S
    - Cost= $b_R + b_S$ + *cost of sorting (R) + cost of sorting(S)* + b_{RES} (p. 26)
  - *In the best case*
    - Cost= $b_R + b_S + b_R \log_2 b_R + b_S \log_2 b_S + b_{RES}$

33

Sort-merge join

- Example: DEPARTMENT $\bowtie_{MGRSSN=SSN}$ EMPLOYEE
  - Suppose primary index exists on SSN and thus the EMPLOYEE is physically sorted by SSN,
  - by contrast the DEPARTMENT is not sorted by its secondary index MGRSSN

Cost= $b_E + b_D + b_D \log_2 b_D$

$= 2000 + 10 + 10 \log_2 (10)$

$=2010 + 10 * 3.321928$

≈ 2043.2

34

Hash-join

- J4. **Hash-join:** The records of files R and S are both hashed to the same hash file, using the same hashing function on the join attributes A of R and B of S as hash keys. A single pass through the file with fewer records (say, R) hashes its records to the hash file buckets ***(called partitioning phase)***. In a second phase ***(called probing or joining phase)*** a single pass through the other file (S) then hashes each of its records to the appropriate bucket, where the record is combined with all matching records from R.
- Cost analysis of partition hash join:
  1. Reading and writing each record from R and S during the partitioning phase: $(b_R + b_S)$, $(b_R + b_S)$
  2. Reading each record during the joining phase: $(b_R + b_S)$
  3. Writing the result of join: b_{RES}

Total Cost: $3^* (b_R + b_S) + b_{RES}$

35

Cost of writing the result file after joining

- **Join selectivity (js)**

  $js = | (R \bowtie_C S) | / | R \times S | = | (R \bowtie_C S) | / (|R| * |S|)$

  If condition C does not exist, js = 1;

  If no tuples from the relations satisfy condition C, js = 0;

  Usually, **0 <= js <= 1**;

  If C has the form of : R.A = S.B

  .If A is a key of $R \Longrightarrow |(R \bowtie_C S)| \leq |S| \Longrightarrow js \leq 1/|R|$

  .If B is a key of $S \Longrightarrow |(R \bowtie_C S)| \leq |R| \Longrightarrow js \leq 1/|S|$
- **Size of the result file after join operation**

  $| (R \bowtie_C S) | = js * |R| * |S|$
- **The cost of writing the result file after join operation**

  $b_{RES} = (js * |R| * |S|)/bfr_{RES}$

36

Join cost estimation example

- **EMPLOYEE $\bowtie_{DNO=DNUMBER}$ DEPARTMENT**
- EMPLOYEE
  - r_E=10,000 b_E=2000 bfr_E=5
  - Secondary index on DNO: X_{DNO}=2, b_{I1DNO}=4, d_{DNO}=125, S_{DNO}=80
- DEPARTMENT
  - r_D=125, b_D=13
  - Primary index on DNUMBER: $X_{DNUMBER}$= 1
- Blocking factor for resulting file: bfr_{ED}=4
- Estimate the worst case costs for the join operation using J1 and J2.

Join cost estimation example

$$js = \left(1 / |DEPARTMENT|\right) = 1/125$$ as DNUMBER is a key of DEPARTMENT

1. Using J1 with EMPLOYEE as outer loop
   $CJ1 = b_E + (b_E * b_D) + (js * r_E * r_D) / bfr_{ED}$
   $= 2000 + (2000 * 13) + (1/125 * 10000 * 125)/4 = 30500$

2. Using J1 with DEPARTMENT as outer loop
   $CJ1a = b_D + (b_D * b_E) + (js * r_E * r_D) / bfr_{ED}$
   $= 13 + (2000 * 13) + (1/125 * 10000 * 125)/4 = 28513$

3. Using J2 with EMPLOYEE as outer loop
   $CJ2 = b_E + (r_E * (X_{DNUMBER} + 1)) + (js * r_E * r_D) / bfr_{ED}$
   $= 2000 + (10000 * 2) + (1/125 * 10000 * 125)/4 = 24500$

4. Using J2 with DEPARTMENT as outer loop
   $CJ2a = b_D + (r_D * (X_{DNO} + S_{DNO})) + (js * r_E * r_D) / bfr_{ED}$
   $= 13 + 125 * (2 + 80) + (1/125 * 10000 * 125)/4 = 12763$

Case 4 has the lowest cost and has to be chosen

PROJECT operation

Algorithm for PROJECT operation $\pi_{<\text{attribute list}>}(R)$

1. If <attribute list> has a key of relation R, extract all tuples from R with only the values for the attributes in <attribute list>.
2. If <attribute list> does NOT include a key of relation R, duplicated tuples must be removed from the results.

- **Methods to remove duplicate tuples** can be implemented via hashing or sorting.
  - On sorting duplicates will become adjacent to each other, and all but one set of duplicates can be deleted.
  - *Optimization:* duplicates can be deleted during run generation as well as at intermediate merge steps in external sort-merge.
  - Hashing is similar – duplicates will come into the same bucket.

39

SET operations

Algorithm for SET operations

- **Set operations**: UNION, INTERSECTION, DIFFERENCE and CARTESIAN PRODUCT.
- **CARTESIAN PRODUCT** of relations R and S include all possible combinations of records from R and S. The attribute of the result include all attributes of R and S.
- Cost analysis of CARTESIAN PRODUCT
  - If R has n records and j attributes and S has m records and k attributes, the result relation will have n*m records and j+k attributes.
  - CARTESIAN PRODUCT operation is very expensive and should be avoided if possible.

40

SET operations

- **UNION**
  1. Sort the two relations on the same attributes.
  2. Scan and merge both sorted files concurrently, whenever the same tuple exists in both relations, only one is kept in the merged results.
- **INTERSECTION**
  1. Sort the two relations on the same attributes.
  2. Scan and merge both sorted files concurrently, keep in the merged results only those tuples that appear in both relations.
- **DIFFERENCE** R-S
  1. Sort the two relations on the same attributes.
  2. keep in the merged results only those tuples that appear in R but not in S.

Dr. Joseph Zalaket

41

Aggregate functions

Implementing Aggregate functions:

- **Aggregate functions**: MIN, MAX, SUM, COUNT and AVG
- Options to implement aggregate functions are : Table Scan or Index table traversing.
- Example

```
SELECT   MAX (SALARY)
FROM     EMPLOYEE;
```

  - If an (ascending) index on SALARY exists for the employee relation, then the optimizer could decide on traversing the index for the largest value,
    - MAX: rightmost position in each index node from the root to the rightmost leaf
    - MIN: leftmost position is followed from the root to the leftmost leaf.
  - Otherwise table scan to find the highest salary value (or lowest for MIN)

Dr. Joseph Zalaket

42

Aggregate functions

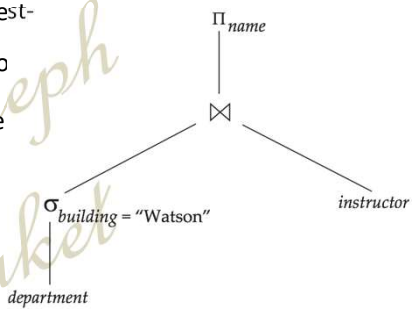
- SUM, COUNT and AVG
  - For a **dense index** (each record has one index entry): apply the associated computation to the values in the index file.
  - For a **non-dense index**: actual number of records associated with each index entry must be accounted for
- With GROUP BY: the aggregate function must be applied separately to each group of tuples.
  - If a clustering index on the group attribute exists use it otherwise, use sorting or hashing on the group attributes, to partition the file into the appropriate groups;
  - Computes the aggregate function for the tuples in each group.

Combining Operations

- We have seen algorithms for individual operations
- Alternatives for evaluating an entire expression tree
  - **Materialization**: generate results of an expression whose inputs are relations or are already computed, materialize (store) it on disk. Repeat.
  - **Pipelining**: pass on tuples to parent operations even as an operation is being executed

Materialization

- **Materialized evaluation:** evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., in figure below, compute and store
  $\sigma_{building="\text{Watson}"}(department)$
  then compute and store its join with *instructor,* and finally compute the projection on *name.*
- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be *quite high*



Dr. Joseph Zalaket

45

Pipelining

- **Pipelined evaluation :** evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of
  $\sigma_{building="\text{Watson}"}(department)$
  - instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., External sort, hash-join.

Dr. Joseph Zalaket

46

Heuristics optimization

- Process for heuristics optimization
  1. The parser of a high-level query generates an *initial internal representation*;
  2. Apply heuristics rules to optimize the internal representation.
  3. A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.
- The main heuristic is to apply first the operations that reduce the size of intermediate results.

  E.g., Apply unary operations such as SELECT and PROJECT before applying the binary operations such as JOIN.

47

Internal query representation

- **Query tree**: a tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as *leaf nodes* of the tree, and represents the relational algebra operations as *internal nodes*.
- An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation.
- **Query graph**: a graph data structure that corresponds to a relational calculus expression. It does not indicate an order on which operations to perform first. There is only a single graph corresponding to each query.

48

Query decomposition

- ***Query decomposition*** is the first step of query processing that transforms a relational calculus query into a relational algebra query.
- The steps of query decomposition are:
  1. Normalization
     - Transform the query into a normal form to ease its manipulation
  2. Analysis
     - detect and reject incorrect queries
  3. Simplification
     - eliminate redundancies
  4. Rewriting
     - Rewrite the query in relational algebra

Dr. Joseph Zalaket

49

DB schema for running examples

EMPLOYEE (SSN, FNAME, MINIT, LNAME, TITLE, BDATE, ADDRESS, SEX, SALARY, SUPERSSN, DNO)

DEPARTMENT (DNUMBER, DNAME, MGRSSN,MGRSTARTDATE)

PROJECT (PNUMBER, PNAME, PLOCATION, DNUM)

WORKS_ON (ESSN, PNO, HOURS)

Dr. Joseph Zalaket

50

Normalization

1. **Normalization**:
   - transform the query into a normalized form to facilitate further processing.
   - There are 2 possible normal forms:
     - The CNF (conjunctive normal form) is a conjunction (^ predicate) of disjunction (v predicates):
       $(P_{11} \vee P_{12} \vee .... \vee P_{1n}) \wedge ..... \wedge (P_{m1} \vee P_{m2} \vee .... \vee P_{mn})$
     - The DNF (disjunctive normal form):
       $(P_{11} \wedge P_{12} \wedge ..... \wedge P_{1n}) \vee .... \vee (P_{m1} \wedge P_{m2} \wedge .... \wedge P_{mn})$

Dr. Joseph Zalaket

51

Normalization

- Example:

```
SELECT Fname, Lname
FROM   Employee, Works_on
WHERE Employee.SSN=Works_on.ESSN
        And Works_on.PNo='P1'
        And (Works_on.Hours =12 OR Works_on.Hours =24);
```

- In CNF:
  Employee.SSN=Works_on.ESSN $\wedge$ Works_on.PNo='P1' $\wedge$ (Works_on.Hours =12 $\vee$ Works_on.Hours =24)
- In DNF
  (Employee.SSN= Works_on.ESSN $\wedge$ Works_on.PNo='P1' ' $\wedge$ Works_on.Hours =12)
  $\vee$
  (Employee.SSN= Works_on.ESSN $\wedge$ Works_on.PNo='P1' ' $\wedge$ Works_on.Hours =24)
- Remarks:
  - The first (CNF) is better, because in DNF treating the 2 conjunctions independently may lead to redundant work. The CNF is more practical since query qualifications include more AND than OR predicates.
  - OR's are mapped into union, AND's are mapped into join or selection

Dr. Joseph Zalaket

52

Analysis

2. Analysis:
   - Query analysis enables the rejection of normalized queries which are type incorrect or semantically incorrect.
   - *Type Incorrect*:
     - If an attribute or a relation name is not defined in the global schema: ex: SELECT address FROM Works_on;
     - If a wrong type is applied.
       ex: SELECT * FROM Employee WHERE Lname > 500;
   - *Semantically Incorrect*: A query is semantically incorrect if some of its components do not contribute to the generation of the result. (A query graph (join graph) can be used to determine the semantic correctness).

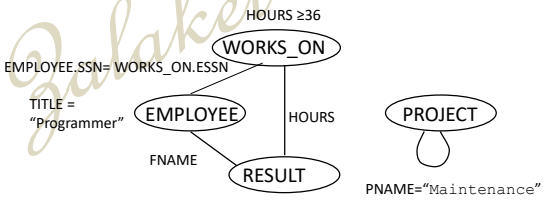
Dr. Joseph Zalaket

Analysis

```
SELECT   FNAME,HOURS
FROM     EMPLOYEE, WORKS_ON, PROJECT
WHERE    EMPLPYEE.SSN = WORKS_ON.ESSN
AND      PNAME = "Maintenance"
AND      HOURS ≥ 36
AND      TITLE = "Programmer"
```

the query graph is not connected (No join condition with PROJECT and it does not participate to the result)→ the query is wrong

Query graph



Dr. Joseph Zalaket

Simplification

3. Simplification

- Elimination of redundancy: Redundant query qualifications (where clauses) can be eliminated using the idempotency rules:
  1. $p \wedge p \Leftrightarrow p$
  2. $p \vee p \Leftrightarrow p$
  3. $p \wedge true \Leftrightarrow p$
  4. $p \vee false \Leftrightarrow p$
  5. $p \wedge false \Leftrightarrow false$
  6. $p \vee true \Leftrightarrow true$
  7. $p \wedge \neg p \Leftrightarrow false$
  8. $p \vee \neg p \Leftrightarrow true$
  9. $p1 \wedge (p1 \vee p2) \Leftrightarrow p1$
  10. $p1 \vee (p1 \wedge p2) \Leftrightarrow p1$

55

Simplification

```
SELECT      TITLE
FROM        EMPLOYEE
WHERE       EMPLOYEE.FNAME = "Joe"
OR          (NOT(TITLE = "Programmer")
AND          (TITLE = "Programmer"
OR           TITLE = "Computer Eng.")
AND         NOT(TITLE = "Computer Eng."))
```

- Applying the idempotency rules the query can be simplified into:

$$\Downarrow$$

```
SELECT      TITLE
FROM        EMPLOYEE
WHERE       FNAME = "Joe"
```


56

Rewriting

4. Rewriting:

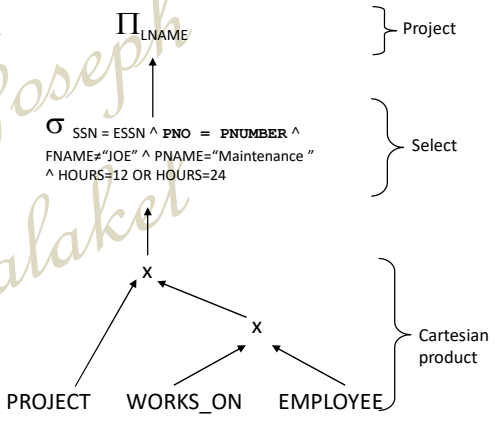
- The last step of query decomposition is to rewrite the query in relational algebra. This is done in 2 steps:
  - Transformation from relational calculus to relational algebra.
  - Reconstruction of the relational algebra query to improve performance (using operator tree).

Rewriting - Transformation

- **Example :**

```
SELECT LNAME
FROM   EMPLOYEE,
WORKS_ON, PROJECT
WHERE    SSN = ESSN
AND      PNO = PNUMBER
AND      FNAME ≠ "Joe"
AND      PNAME =
"Maintenance"
AND     (HOURS= 12 OR
HOURS= 24)
```

- Transform the relational calculus query into a relational algebra query using a query tree
- Remark*: many different trees may be found equivalent to this one during transformation*.



Rewriting - Reconstruction

- Reconstruction rules:
  1. Commutativity (binary operators)
     - $R \; op \; S \Leftrightarrow S \; op \; R$, for $op \in \{\times, \bowtie, \cup, \cap\}$
  2. Associativity (binary operators)
     - $(R \; op \; S) \; op \; T \Leftrightarrow R \; op \; (S \; op \; T)$, for $op \in \{\times, \bowtie, \cup, \cap\}$
  3. Idempotency (unary operators)

     Assume: R = $\{A_1, A_2, \ldots, A_n\}$ \ $A' \subseteq R$, $A'' \subseteq R$ and $A' \subseteq A''$
     - $\Pi_{A'}(\Pi_{A''}(R)) \Leftrightarrow \Pi_{A'}(R)$
     - $\sigma_{p_1(A_1)}(\sigma_{p_2(A_2)}(R)) = \sigma_{p1(A_1) \wedge p_2(A_2)}(R)$, where p_i is a predicate.

Rewriting - Reconstruction

4. Commutativity of selection with projection
   - $\Pi_{A1,\ldots,Ai}(\sigma_{p_j(A_j)}(R)) \Leftrightarrow \Pi_{A1,\ldots,Ai}(\sigma_{p_j(A_j)}(\Pi_{A1,\ldots,Ai,\,Aj}(R)))$, where $A_j \notin \{A_1, \ldots, A_i\}$
5. Commutativity of selection with binary operators
   - $\sigma_{p(Ai)}(R \times S) \Leftrightarrow (\sigma_{p(Ai)}(R)) \times S$
   - $\sigma_{p(A_i)}(R \bowtie_{(A_j,B_k)} S) \Leftrightarrow (\sigma_{p(A_i)}(R)) \bowtie_{(A_j,B_k)} S$, where $R = \{A_1, A_2, \ldots, A_n\}$ and $S = \{B_1, B_2, \ldots, B_m\}$
   - $\sigma_{p(A_i)}(R \cup T) \Leftrightarrow \sigma_{p(A_i)}(R) \cup \sigma_{p(A_i)}(T)$
   - $\sigma_{p(A_i)}(R \cap T) \Leftrightarrow \sigma_{p(A_i)}(R) \cap \sigma_{p(A_i)}(T)$, where $A_i \in R$, $A_i \in T$, R and T *are union compatible.*

Rewriting - Reconstruction

6. Commutativity of projection with binary operators
   - $\Pi_C(R \times S) \Leftrightarrow \Pi_{A'}(R) \times \Pi_{B'}(S)$
   - $\Pi_C(R \bowtie_{(A_i,B_j)} S) \Leftrightarrow \Pi_{A'}(R) \bowtie_{(A_i,B_j)} \Pi_{B'}(S)$
     if $A_i \in A'$ and $B_j \in B'$,
     $\Leftrightarrow \Pi_{A',Ai}(R) \bowtie_{(A_i,B_j)} \Pi_{B',Bj}(S)$,otherwise.

     , where $C = A' \cup B'$ and $A' \subseteq R$, $B' \subseteq S$
   - $\Pi_C(R \cup S) \Leftrightarrow \Pi_C(R) \cup \Pi_C(S)$
   - $\Pi_C(R \cap S) \Leftrightarrow \Pi_C(R) \cap \Pi_C(S)$

     , where R and S are union compatible

61

Rewriting - Reconstruction

7. Converting a selection, Cartesian product (σ, x) sequence into Join ⋈: If the condition c of a σ that follows a x Corresponds to a join condition such that: c={A_i=B_j} and $A_i \in$ R and $B_j \in$ S, convert the (σ, x) sequence into a ⋈ as follows:
   $(\sigma_C (R \times S)) \Leftrightarrow (R \bowtie_{(A_i,B_j)} S)$

62

Rewriting - Reconstruction

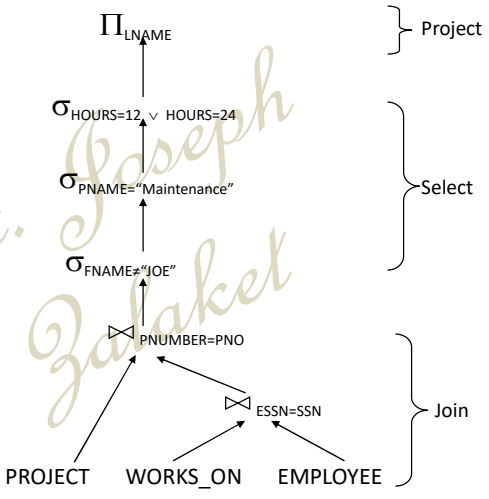
- The application of the 7 rules of reconstruction may generate multiple equivalent trees. Choosing the best of them during the optimization phase is unrealistic.
- These rules can be used to reconstruct the tree by eliminating unnecessary operators in 4 different ways:

i. Separation of unary operations
ii. Group the unary operations by relation to be accessed once.
iii. Commute unary operations with binary operations, so that operations like SELECT can be done first. (order unary operations to be done as much as possible before binary operations).
iv. Order the binary operations (used in query optimization)

63

Rewriting - Reconstruction

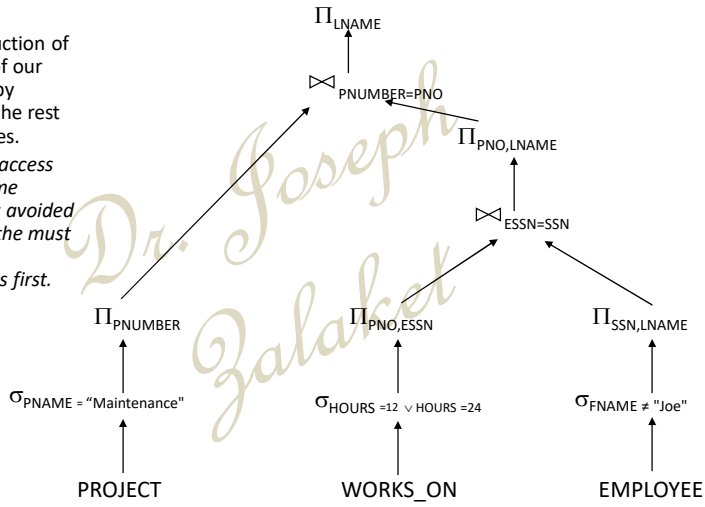
- Reconstruction of the tree of our example by applying rules 3 and 7 (cartesian products are avoided and replaced by joins)



64

Rewriting - Reconstruction

- Reconstruction of the tree of our example by applying the rest of the rules.
- *repeated access to the same relation is avoided by doing the must selective operations first.*



Dr. Joseph Zalaket

65

Query Optimization

- After query rewriting in relational algebra from relational calculus. The query optimization refers to the process of producing a *query execution plan* which minimize an objective cost function.
- Query execution plans are abstracted by means of operator trees.
- Definition : A *search space* is the set of equivalent operator trees that can be produced using transformation rules.
- For *N* relations, there are *O(N!)* equivalent operator trees that can be obtained by applying commutativity and associativity rules.

Dr. Joseph Zalaket

66

Query Optimization

- Query optimizers restrict the size of the search space by using heuristics such as:
  - Perform selection and projection first on the base relations before applying JOIN.
  - Avoid Cartesian products that are not required by the query
  - …
- Another restriction can be applied by using linear join trees instead of bushy join trees, which can reduce the search space to $O(2^N)$.
- A linear join tree has at least one operand of each operation as a base relation.
- A bushy join tree has operators with no base relations as operands.



67

Heuristics Optimization

- **Summary of Heuristics Optimization:**

1. The main heuristic is to apply first the operations that reduce the size of intermediate results.
2. Perform select operations as early as possible to reduce the number of tuples and perform project operations as early as possible to reduce the number of attributes. (This is done by moving select and project operations as far down the tree as possible.)
3. The select and join operations that are most restrictive should be executed before other similar operations. (This is done by reordering the leaf nodes of the tree among themselves and adjusting the rest of the tree appropriately.)

68

Example

Sailors (*sid*, *sname*, *rating*, *age*)
Reserves (*sid*, *bid*, *day*, *rname*)
Boat (*bid*, *bname*, *capacity*)

- With the following information:
  - Reserves:
    - R_R = 40 bytes, b_R =1000 blocks, bfr_R = 100 tuples per block.
  - Sailors:
    - R_S = 50 bytes, b_S =500 blocks, bfr_S = 80 tuples per block, d_{rating}=10.
  - Boat:
    - r_{BO} = 100 tuples.
  - Memory buffer:
    - n_B= 5 blocks

Dr. Joseph Zalaket

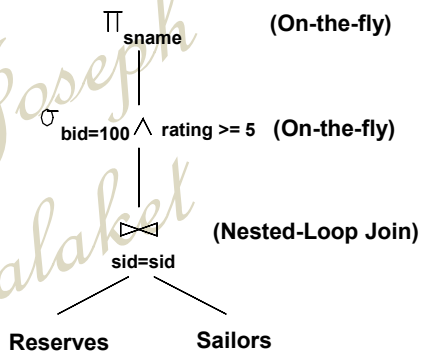
69

Example

SELECT S.sname
FROM Reserves R, Sailors S
WHERE R.sid=S.sid AND
R.bid=100 AND S.rating>=5

- Applying the nested-loop join and ignoring the result writing cost. Selection and projection can be done on-the-fly, that means no additional cost is required.
- Cost = $b_S * b_R + b_{S=}$ 500+500*1000 = 500500 (the worst plan)

Plan:

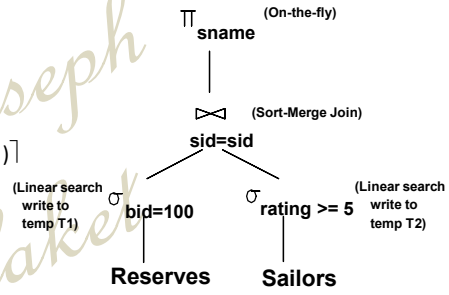


Dr. Joseph Zalaket

70

Example

- Linear search (S1) of Reserves (c11=b_R =1000) + write temp T1 (c12=10 blocks, if we have 100 boats, (1000/100) uniform distribution). c1=c11+c12=1010
- Linear search (S1) Sailors (c21=b_S =500) + write temp T2 (c22=250 blocks, if we have 10 ratings). c2=c21+c22=750
- Apply Sort-merge on T1 and T2 separately before joining C = $2b*\lceil 1 + (\log_{nB-1} \lceil (b/n_B) \rceil) \rceil$
- Sort T1 (c3= 2*10*(1+1)=40), sort T2 (c4= 2*250 *(1+3)=2000),
- Merge T1 and T2 by applying sort-merge join C= $b_{T1} + b_{T2}$
- (c5= 10+250=260)
- **Total Cost=c1+…+c5 = 4060 blocks I/Os.**
- If we used **hash join**, join cost c6= 3*(b_{T1} + b_{T2}) = 780 and **Total cost = c1+c2+c6= 2540.**
- *What if we take into account the Primary key sid on Sailors with x=3? Projection first?*



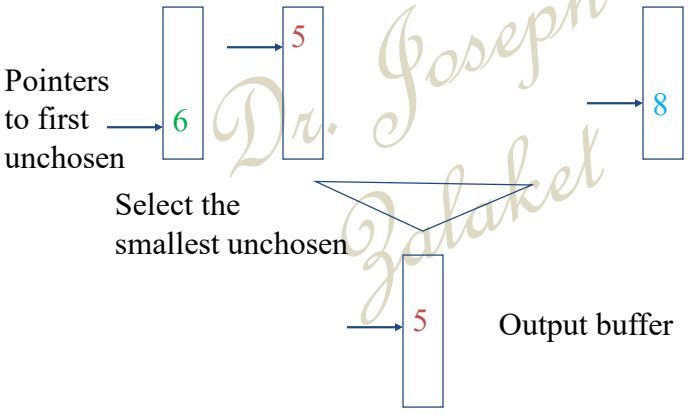
Dr. Joseph Zalaket

71

Merge-sort

Phase 2

- Use one buffer for each of the sorted sublists and one buffer for output.



Dr. Joseph Zalaket

72